

Ânima Educação

StudyFlow

Relatório técnico de requisitos e qualidade

Disciplina: Gestão e Qualidade de Software **Projeto:** A3 Prático / Lista de Exercícios III **ODS:** ODS 4 - Educação de Qualidade

Integrantes

- João Vitor Alves Rodrigues - RA 32513480
- Rafael Luiz Ferreira de Souza - RA 32511503
- Pietro Cardoso de Oliveira - RA 32515280
- Gustavo Henrique Ramos Gomes - RA 325141430
- Pedro Henrique Martins - RA 325130235

Sumário

- 1. Visão geral e requisitos
- 2. Modelagem, arquitetura e classes
- 3. Versionamento e garantia da qualidade
- 4. Fluxo crítico e conclusão

1. Visão geral e requisitos

Problema e solução

Estudantes concentram tarefas, leituras, avaliações e entregas em canais diferentes. A falta de uma visão única de prioridades e prazos aumenta atrasos, retrabalho e desorganização.

O StudyFlow centraliza atividades acadêmicas em um dashboard. Cada tarefa possui título, disciplina, prazo, prioridade e status. O público-alvo são estudantes de graduação. A solução se relaciona à ODS 4 - Educação de Qualidade ao apoiar autonomia e continuidade do aprendizado.

Matriz de requisitos

ID	Tipo	Descrição	Prioridade	Critério de aceite
RF-01	RF	Cadastrar tarefa.	Alta	A tarefa aparece como pendente.
RF-02	RF	Listar tarefas por pendência e prazo.	Alta	Pendentes aparecem primeiro.
RF-03	RF	Concluir ou reabrir tarefa.	Alta	Status e contagem atualizam.
RF-04	RF	Filtrar tarefas.	Média	Apenas itens correspondentes aparecem.
RF-05	RF	Exibir indicadores semanais.	Média	Dashboard mostra progresso.
RF-06	RF	Persistir via API Python.	Alta	Dados permanecem após reinício.
RNF-01	RNF	Responsividade.	Alta	Funciona em 360px, 768px e desktop.
RNF-02	RNF	Desempenho.	Média	Carrega em até 2 segundos.
RNF-03	RNF	Segurança de entrada.	Alta	API rejeita dados inválidos.
RNF-04	RNF	Manutenibilidade.	Média	Há factory, banco isolado e testes.
RNF-05	RNF	CI.	Baixa	PR executa testes e build.

2. Modelagem, arquitetura e classes

Modelo de dados

A entidade Task possui id, title, subject, due_date, priority, status e created_at. O desenvolvimento usa SQLite local; a integração pode usar MySQL por STUDYFLOW_DATABASE_URL. O schema está em backend/schema.mysql.sql.

```
erDiagram
    TASK {
        int id PK
        string title
        string subject
        string due_date
        string priority
        string status
        datetime created_at
    }
```

Diagrama de classes simplificado

```
classDiagram
    class Task {
        +int id
        +string title
        +string subject
        +string due_date
        +Priority priority
        +TaskStatus status
        +complete()
        +reopen()
    }
    class TaskRepository {
        +list(status)
        +create(data)
        +update_status(id, status)
        +delete(id)
    }
    class TaskAPI {
        +GET /api/tasks
        +POST /api/tasks
        +PATCH /api/tasks/id
        +DELETE /api/tasks/id
    }
    TaskAPI --> TaskRepository
    TaskRepository --> Task
```

Arquitetura

```
flowchart LR
    Browser[React + Vite] --> JSON_HTTP[JSON HTTP]
    API[Flask REST API] --> JSON_HTTP
    API --> DB[(SQLite ou MySQL)]
    CI[GitHub Actions] --> Browser
    CI --> API
```

Regras de negócio

1. Título obrigatório.
2. Prioridade: Alta, Média ou Baixa.
3. Status: pending ou done.
4. Pendentes antes das concluídas.
5. Tarefa concluída pode ser reaberta.

3. Versionamento e garantia de qualidade

GitFlow: main estável, develop para integração e feature/nome-curto para funcionalidades. Commits semânticos e Pull Requests com CI verde e revisão.

Cenários de QA

QA-01 - Cadastro: dados válidos geram tarefa pendente.

QA-02 - Conclusão: controle atualiza status, texto e contagem.

QA-03 - API inválida: POST sem título retorna HTTP 400.

QA-04 - Responsividade: 360px, 768px e 1440px não apresentam sobreposição.

4. Fluxo crítico e conclusão

1. Abrir dashboard.
2. Conferir prioridades.
3. Criar tarefa.
4. Salvar no banco.
5. Executar atividade.
6. Marcar como concluída.

O código está funcional, testado e documentado. O banco inicia vazio por padrão. A entrega inclui código Python, frontend responsivo, persistência configurável, testes unitários, CI/CD, requisitos, Word, PDF e PowerPoint.

Pendências externas

GitHub, vídeo pitch e WayHub são entregas posteriores. Também devem ser confirmados o ensaio de 10 a 15 minutos e a participação de todos.